

20. Une table gravée : dessiner un niveau

Le niveau est une liste de nombres ; deux boucles en font un paysage.

1. Ce qu'il faut comprendre

`const uint8_t HAUTEURS[] = { ... };` est un tableau **constant** : il est **gravé dans la cartouche**, comme le programme. Il ne prend **aucun octet** des 8 Ko de mémoire de travail — et il ne peut plus changer. C'est la place naturelle des **données d'un niveau**.

Les crochets vides `[]` laissent le compilateur compter les valeurs. `sizeof(HAUTEURS)` rend ce nombre : ajouter une colonne ne demande de changer **rien d'autre**.

Le dessin combine tout ce qu'on a vu : **pour chaque colonne** (le `for` du dehors), **on empile autant de blocs que la hauteur** (le `for` du dedans, dont la limite vient du tableau). C'est le triangle du chapitre 3 — mais la limite est lue dans des données, au lieu d'être calculée.

`17 - h` : l'écran compte ses lignes **vers le bas**, alors qu'on empile **vers le haut**. La ligne 17 est tout en bas ; un bloc de hauteur 0 s'y pose, le suivant sur la 16...

C'est ainsi que sont faits les niveaux de beaucoup de jeux : **le code ne change pas, seules les données changent**. Un nouveau niveau, c'est un nouveau tableau.

À toi : dessine un second niveau en changeant seulement les nombres.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **755 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         150 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← ecran() – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
AdresseCase    10 o ← texte() – trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles   85 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         304 o ← CopierTuiles – 19 lettres sur 43
"##"           1 o ← texte() – un texte gravé
"COLONNES:"    9 o ← texte() – un texte gravé
"BLOCS:"       6 o ← texte() – un texte gravé
HAUTEURS_0     20 o ← main() – une table gravée
```

Lettres gravées : BCELNOS0123456789:##

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte()  × 3    48 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
ecran()  × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
nombre() × 2    20 o sur place, dans main() + EcrireNombre (33 o)
image()  × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : dessine un paysage de collines à partir d'une liste de hauteurs.
//
// Ce qui est nouveau : const devant un tableau.
// const = « constant » : le tableau ne changera jamais. Il est gravé dans
// la cartouche et ne prend pas de place dans la petite mémoire de travail.
// [] vide : le compilateur compte lui-même les valeurs (ici 20).

#include <ecran>    // éteint ou rallume l'écran
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres

```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — HAUTEURS

```

const uint8_t HAUTEURS[] = {      // La hauteur de chaque colonne, de gauche à droite :
    2, 2, 3, 5, 5, 4, 2, 1, 1, 3,  // colonnes 0 à 9
    6, 6, 3, 2, 2, 4, 4, 2, 1, 1,  // colonnes 10 à 19
};

```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {                      // Le jeu commence ici.
    ecran(0);                     // Dessin écran éteint.
    uint8_t blocs = 0;            // Combien de # posés en tout.

    for (uint8_t colonne = 0; colonne < sizeof(HAUTEURS); colonne++) { // colonne = 0 à 19
        for (uint8_t h = 0; h < HAUTEURS[colonne]; h++) { // h = 0 jusqu'à la hauteur - 1
            texte(colonne, 17 - h, "#"); // On empile depuis le BAS : la ligne 17 est la
                                          // dernière de l'écran. h = 0 → ligne 17, h = 1 → 16...
                                          // Colonne 3 (hauteur 5) : lignes 17, 16, 15, 14, 13.

            blocs++;                // un # de plus.
        }
    }

    texte(1, 1, "COLONNES:"); nombre(12, 1, sizeof(HAUTEURS)); // 020
    texte(1, 2, "BLOCS:");    nombre(12, 2, blocs);           // la somme des hauteurs : 59

    ecran(1);                  // On rallume : le paysage apparaît.

    while (true) {             // La boucle du jeu.
        image();
    }
}

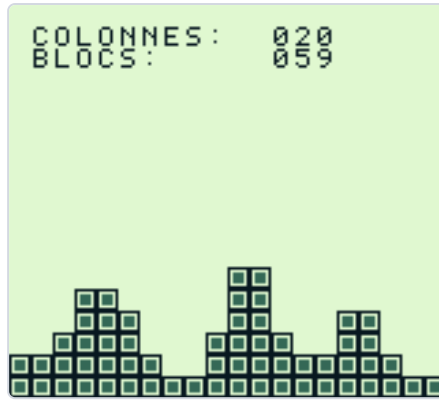
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un paysage de collines au bas de l'écran, 20 colonnes, 59 blocs.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ chaque colonne a la hauteur lue dans le tableau
- ✓ sizeof compte 20 colonnes
- ✓ 59 blocs posés

← [19. Parcourir : le total, le plus grand, le plus petit](#)

[sommaire](#)

[21. Chercher et changer une case](#) →